EXPRESS JS & MYSQL

INDEX.JS

```
// REST API => curd - create , update, read,, delete

const express = require("express")

const app = express();
const userRoutes = require("./routes/userRoutes")


app.use(express.json());

app.use(userRoutes);


app.listen(3000, ()=>{
    console.log("Server is up and running");
})
```

DB.JS

```
const mysql = require("mysql2/promise")


const pool = mysql.createPool({
    host:"localhost",
    user:"root",
    password:"raj123",
    database:"raviraj"
})


module.exports = pool;
```

userRoutes.js

```
const express = require("express");

const router = express.Router();
```

```
const { getUser, addUser, updateUser, deleteUser } =
require("../controller/userController");

router.get("/user", getUser);

router.post("/user" , addUser);

router.put("/user/:id", updateUser);

router.delete("/user/:id",deleteUser)

module.exports = router;
```

userController.js

```
const pool = require("../db");

let user = [];


// const newUser = {
//   id: 1,
//   name: "Raviraj",
//   password: "ravi123",
// };
// user.push(newUser);

const getUser = async (req, res) => {

  const [rows] = await pool.query("SELECT * FROM user");
  res.json(rows);
  // res.json(user);
};

const addUser = async (req, res) => {
  const { name, password ,age} = req.body;

  await pool.query("INSERT INTO user(name, password, age)
VALUES(?,?,?)",[name, password, age])
  // const newUser = {
  //   id: user.length + 1,
  //   name,
  //   password,
  // };
  // user.push(newUser);

  res.send("User created successfully");
```

```
};

const updateUser = async (req, res) => {
  const id = Number(req.params.id);
  const { name, password ,age} = req.body;

  await pool.query("UPDATE user SET name = ? , password =? , age = ? WHERE id
= ?", [name, password, age ,id]);

  // const hasToUpdateUser = user.find((user) => user.id === id);

  // hasToUpdateUser.name = name;
  // hasToUpdateUser.password = password;


  res.json("Updated successfully");
};

const deleteUser = async (req,res)=>{
    const id = Number(req.params.id);

    // user = user.filter((user)=> user.id != id);
    await pool.query("DELETE FROM user WHERE id=?",[id]);

    res.json({
        message:"user deelted successfully"
    })
}

module.exports = {
  getUser,
  addUser,
  updateUser,
  deleteUser
};
```

Sql

```
-- DDL => DATA DEFINITION LANGUAGE
CREATE TABLE user(
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    password VARCHAR(100) NOT NULL
)


ALTER TABLE user ADD age INT NOT NULL;
```

```sql
DROP TABLE user;

-- DML = > DATA MANIPULATION LANGUAGE
INSERT INTO user(name, password, age) VALUES("Raviraj","raviraj123", 21);

INSERT INTO user(name, password, age) VALUES("Kicha","kicha23", 20);


UPDATE user SET age = 50  WHERE id = 3;

DELETE FROM user ;


-- DQL => DATA QUERY LANGUAGE
SELECT name,password FROM user WHERE id=4;
```

Theory Notes

1. REST API

- **REST (Representational State Transfer)** is an architectural style for APIs.
- CRUD operations map to HTTP methods:
  - **Create** → `POST`
  - **Read** → `GET`
  - **Update** → `PUT`
  - **Delete** → `DELETE`

2. Express.js

- `express()` creates the server.
- `express.json()` parses JSON request bodies.
- `express.Router()` organizes routes into separate files.
- Routes are defined with methods like `router.get()`, `router.post()`, etc.

3. MySQL Integration

- Using `mysql2/promise` with `createPool()` creates a connection pool.
- Queries are executed with `await pool.query("SQL", [params])`.
- Pooling improves performance by reusing connections.

4. Controller Functions

- **getUser** → fetches all users from DB.
- **addUser** → inserts new user record.
- **updateUser** → updates existing user by ID.

- **deleteUser** → deletes user by ID.
- Each function uses async/await for non-blocking DB calls.

5. SQL Basics

- **DDL (Data Definition Language)** → defines structure (`CREATE`, `ALTER`, `DROP`).
- **DML (Data Manipulation Language)** → modifies data (`INSERT`, `UPDATE`, `DELETE`).
- **DQL (Data Query Language)** → queries data (`SELECT`).
- Example:

sql
```
CREATE TABLE user(
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  password VARCHAR(100) NOT NULL,
  age INT NOT NULL
);
```

6. Error Handling

- Always validate inputs before inserting/updating.
- Return appropriate HTTP status codes:
  - `200 OK` → success
  - `201 Created` → resource created
  - `404 Not Found` → resource not found
  - `400 Bad Request` → invalid input

Practice Tasks

Short Answer

1. What is the difference between `PUT` and `PATCH`?
2. Why is `mysql2/promise` preferred over plain `mysql` in async code?
3. What does `AUTO_INCREMENT PRIMARY KEY` do in SQL?
4. How does `express.json()` help in handling requests?

Coding Exercises

1. Add a new route `/user/:id` (GET) to fetch a single user by ID.
2. Modify `addUser` to validate that `age` must be greater than 0.
3. Add error handling so that `updateUser` returns `404` if the user doesn't exist.
4. Extend the API to include a `PATCH /user/:id` route that updates only one field (e.g., just the name).
5. Create a middleware that logs request method and URL for every request.
6. Add a new column `email` to the `user` table and update the API accordingly.
7. Implement pagination in `getUser()` (e.g., `/user?page=1&limit=5`).